Securing ERC-4337 Paymasters

A Defense Framework Against Abuse and Deposit Drain Attacks

Rahulkumar Rajeshkumar Ankola

Capstone Project | Dr. Desmond Lun | Spring 2026

Abstract

ERC-4337 paymasters are smart contracts that sponsor gas fees on behalf of users, enabling gasless transactions in Account Abstraction architectures. Their open sponsorship model exposes them to deposit drain attacks, spam exploitation, and gas penalty abuse. This paper presents a comparative security analysis of two paymaster implementations deployed on the Ethereum Sepolia testnet: a permissive Baseline Paymaster serving as a control, and a Secured Paymaster enforcing gas caps, contract whitelisting, per-user and per-dapp daily quotas, and cryptographic sponsorship authorization. Attack simulations across two traffic mixes (80/10/10 valid/overcap/non-whitelist and 100% valid) demonstrate that the Secured Paymaster eliminates all simulated exploit costs with a break-even malicious traffic threshold of approximately 17.87%, yielding $1,600/month in projected savings at 100k transactions with 20% attack traffic. A hybrid routing strategy is proposed for production deployments.

RUTGERS
UNIVERSITY | CAMDEN

Index

1. Introduction

1.1 Problem Statement

ERC-4337 introduces Account Abstraction (AA) to Ethereum without requiring protocol-level changes, enabling smart contract wallets that can perform actions previously restricted to Externally Owned Accounts (EOAs). A critical enabler of this architecture is the Paymaster, a smart contract that pays gas fees on behalf of users. While this unlocks powerful user experience improvements such as gasless transactions and ERC-20 token payments for gas, it introduces a high-value attack target: the paymaster's ETH deposit.

Poorly secured paymasters are vulnerable to deposit drain attacks, where adversaries craft malicious UserOperations designed to maximally consume the paymaster's deposit. Attack vectors include submitting operations with inflated gas limits to trigger penalty charges, targeting non-whitelisted contracts to probe validation logic, and flooding the paymaster with spam operations to exhaust daily quotas. In production environments, even a small percentage of malicious traffic can result in significant financial losses for protocol operators.

As of 2026, ERC-4337 is live on Ethereum mainnet and major L2s including Arbitrum, Optimism, Polygon, and Base, with production deployments by Coinbase, Alchemy, Biconomy, and others serving millions of transactions. The financial stakes of paymaster security failures are therefore substantial.

1.2 Objective

This project designs, implements, and empirically evaluates a Secured Paymaster that enforces a multi-layered defense framework against known abuse vectors. The core contributions are:

- A production-grade SecuredPaymaster contract with gas cap enforcement, contract whitelisting, per-user and per-dapp daily quotas, and cryptographic sponsorship authorization.
- A permissive BaselinePaymaster deployed as a controlled benchmark for side-by-side comparison.
- End-to-end attack simulation scripts that inject realistic mixed-traffic scenarios (valid, overcap, non-whitelist operations) against both paymasters on the Sepolia testnet.
- Quantitative cost and performance analysis, including a break-even malicious traffic threshold model and production deployment decision matrix.

The goal is to provide protocol operators with evidence-based guidance on when and how to deploy secured paymaster infrastructure.

2. Background

2.1 ERC-4337 and Account Abstraction

ERC-4337, finalized on Ethereum mainnet in March 2023, achieves Account Abstraction at the application layer by introducing a higher-layer pseudo-transaction called a UserOperation. Rather than submitting transactions directly to the Ethereum mempool, users submit UserOperations to a separate mempool managed by a network of Bundlers. Bundlers aggregate multiple UserOperations into a single on-chain transaction and submit it to a singleton EntryPoint smart contract that orchestrates execution.

This architecture separates transaction validation from execution and introduces pluggable components for each wallet's signature scheme, gas payment, and execution logic. Smart contract wallets conforming to the IAccount interface gain programmable authorization, enabling multi-sig, social recovery, session keys, and other advanced features that would require EOA key management in a traditional setup.

2.2 The ERC-4337 Transaction Flow

A complete UserOperation lifecycle proceeds through the following stages:

1. User constructs a UserOperation struct containing sender, nonce, callData, gas parameters, and paymaster data.
2. UserOperation is submitted to the alt mempool and picked up by a Bundler.
3. Bundler calls EntryPoint.handleOps(), which invokes the Paymaster's validatePaymasterUserOp() during the validation phase.
4. If validation passes, the Paymaster approves sponsorship and pre-funds the EntryPoint for the estimated gas cost.
5. EntryPoint executes the UserOperation's callData on the sender smart account.
6. EntryPoint calls the Paymaster's postOp() to settle actual gas costs, with only 90% of unused pre-fund refunded.

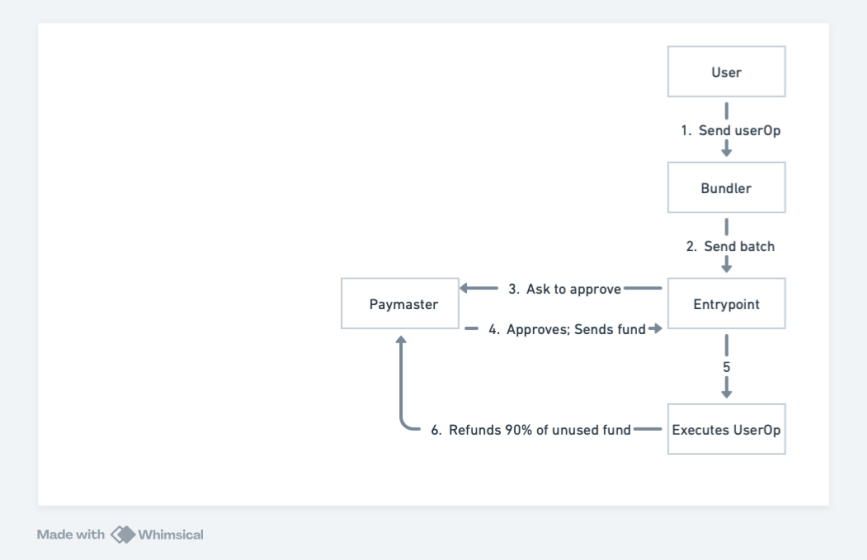

Figure 1. ERC-4337 transaction lifecycle with Baseline Paymaster.

2.3 Paymaster Lifecycle and Economics

Paymasters maintain an ETH deposit at the contract. During validation, the EntryPoint reserves a requiredPreFund amount (computed from the UserOperation's gas limits) from this deposit. If the operation is rejected post-validation by a bundler (e.g., due to simulation failure), the paymaster may incur a penalty charge of up to 10% of the pre-fund as a grieving deterrent defined in EIP-4337. Actual execution costs are settled in postOp, with unused gas returned.

This economic model creates two attack surfaces: (1) the validation phase, where inflated gas limits can trigger large pre-fund reservations and potential penalty charges, and (2) the execution phase, where unlimited sponsorship allows adversaries to drain the deposit through high volumes of operations.

2.4 Production Deployment Context

ERC-4337 paymasters are no longer experimental. Coinbase Smart Wallet, Alchemy's Gas Manager, Biconomy, ZeroDev, and Safe all operate sponsored-transaction infrastructure on Ethereum mainnet and major L2s including Arbitrum, Optimism, Polygon, and Base. Web3 gaming platforms (Immutable, Sequence, Gods Unchained) sponsor millions of in-game micro-transactions to remove wallet friction; DeFi protocols use paymasters to enable ERC-20 gas payment and complex multi-step flows; enterprise providers use them for treasury-controlled corporate wallets. Across these deployments, paymaster deposits collectively hold tens of millions of dollars in ETH, making any systemic vulnerability in paymaster validation logic a high-value target for adversaries.

2.5 Related Work

This project synthesizes and extends prior work on ERC-4337 security from three complementary streams: academic vulnerability analysis, vendor-published security disclosures, and industry best-practice guidance. The literature broadly agrees on the attack surfaces but lacks an empirically-quantified comparison of mitigation strategies — the gap this work addresses.

2.5.1 Academic Analysis of Account Abstraction Security

Boi et al., "Account Abstraction, Analysed" (arXiv:2309.00448), provides an early systematic analysis of ERC-4337 from a security standpoint. The paper argues that moving authorization, gas payment, and execution policy from the protocol layer into smart contracts shifts the security burden onto application developers. The authors highlight that programmability at the paymaster layer is a double-edged sword: it unlocks novel UX patterns but introduces a class of validation-logic bugs that did not exist with protocol-enforced gas payment. Their analysis identifies griefing through gas penalties and unbounded sponsorship as the two primary economic attack surfaces, but does not quantify the cost of these attacks in deployed systems. This work builds on their categorization by implementing both vulnerable and hardened reference paymasters and measuring the actual ETH cost of each attack vector under controlled conditions on Sepolia.

2.5.2 Vendor Security Research on UserOperation Encoding

Alchemy's research team has published technical analyses identifying signature-related vulnerabilities in early ERC-4337 implementations, particularly around how UserOperation hashes are constructed. Their findings show that signing fewer fields than required (for example, omitting callData or initCode from the digest) allows an attacker to substitute the omitted fields after signature,

leading to unauthorized sponsorship. This motivated the present work's design of the sponsorship signature digest, which explicitly binds paymaster contract address, chainId, virtualUserId, validUntil, sponsorshipNonce, and sponsoredMaxCostWei into the signed payload, and enforces strict nonce-replay protection through an on-chain consumed-nonce mapping.

2.5.3 Industry Defensive Patterns

Industry practitioners — including Calibraint, Stackup, and Pimlico — have published architectural guidance advocating quotas, target whitelisting, and gas caps as the minimum defense set for production paymasters. These recommendations are typically presented as design patterns rather than measured controls, and individual operators are left to estimate the cost-benefit tradeoff. The Secured Paymaster implemented in this project translates these recommendations into a concrete, auditable Solidity contract and provides the first empirical break-even analysis (Section 11) tying their cost to observed attack rates.

2.5.4 Position of This Work

Prior literature establishes the threat surface and recommends mitigations qualitatively. This project's contribution is to (a) implement a complete, side-by-side baseline-vs-secured deployment, (b) replay an identical, randomized traffic mix against both implementations, and (c) derive a closed-form break-even threshold (~17.87%) above which the secured design is strictly preferable on both cost and latency. The result is a deployment decision matrix grounded in measured cost, not assumed cost.

3. Threat Model

3.1 Attacker Capabilities

We model an adversary with the following capabilities:

- Ability to create arbitrary smart contract wallet addresses and submit UserOperations to the public alt mempool.
- Knowledge of the paymaster contract address and its ABI, which are publicly visible on-chain.
- Ability to craft UserOperations with arbitrary gas parameters, callData, and paymasterAndData fields.
- No privileged access to the paymaster's signing keys, admin functions, or off-chain infrastructure.
- Ability to execute attacks at high frequency, limited only by Sepolia/mainnet mempool rate limits.

3.2 Attack Surfaces

3.2.1 Gas Penalty Exploitation

Attacker submits UserOperations with inflated gas limits to trigger large pre-fund reservations and bundler penalty charges against the paymaster.

3.2.2 Non-Whitelisted Target Exploitation

Without target whitelisting, a paymaster will sponsor gas for calls to any contract address. An attacker can route operations through arbitrary contracts that may cause execution failures, trigger edge-case

gas behaviors, or simply consume sponsorship budget on operations unrelated to the paymaster operator's intended use case.

3.2.3 Quota Exhaustion / Spam

A single identity floods the paymaster with valid-looking operations, exhausting the daily sponsorship budget and denying service to legitimate users.

3.2.4 Signature Replay

If a paymaster uses off-chain authorization signatures but does not enforce nonce uniqueness or expiry, an attacker who intercepts or guesses a valid signature can replay it across multiple operations, bypassing quota and authorization checks.

3.3 Out of Scope

This analysis does not consider protocol-level attacks against the EntryPoint contract itself, bundler censorship attacks, or attacks requiring compromise of the paymaster operator's signing key.

4. Baseline Paymaster

4.1 Design

The BaselinePaymaster is an intentionally permissive implementation that extends the OpenZeppelin-compatible BasePaymaster from @account-abstraction/contracts. It is designed to represent a naive "sponsor everything" approach that an inexperienced developer might deploy, serving as a controlled benchmark for comparison.

The function accepts the PackedUserOperation, userOpHash, and requiredPreFund arguments but ignores all of them, returning an empty context and a zero validationData (indicating unconditional approval). The postOp function is similarly a no-op, performing no cost accounting after execution.

4.2 Limitations

The baseline design exhibits the complete vulnerability profile described in the Threat Model:

- No target validation: the paymaster will sponsor operations directed at any contract or EOA address.
- No identity or authorization checks: any sender can submit operations for sponsorship without any form of credential.
- No replay protection: there are no nonces or expiry mechanisms on the sponsorship.
- No quota or rate limiting: a single attacker can exhaust the paymaster deposit in a single burst.
- No gas cap enforcement: UserOperations with arbitrarily inflated gas limits are approved unconditionally.

These limitations make the BaselinePaymaster unsuitable for any production deployment, but it provides a clean empirical baseline for quantifying the cost impact of each security control added by the SecuredPaymaster.

5. Attack Vectors

5.1 Spam and Griefing

In a spam attack, the adversary submits a high volume of UserOperations that are technically valid (they pass validation) but economically wasteful. The paymaster sponsors each operation at the normal gas cost, draining the deposit without providing any legitimate service. In a griefing variant, the operations are designed to fail after validation (e.g., by calling a contract that always reverts), maximizing the bundler's penalty charges against the paymaster. Against the BaselinePaymaster, both attack types succeed unconditionally since there are no identity checks, quotas, or target restrictions.

5.2 Gas Penalty Exploitation

The gas penalty attack is a more targeted form of deposit drain. The attacker crafts UserOperations with gas limits set at multiples of the legitimate cost — in our simulation, 5x the normal profile. The large pre-fund reservation triggers a correspondingly large penalty if the operation fails post-validation. Against the BaselinePaymaster, overcap operations proceed to on-chain execution and incur significantly higher costs:

Scenario	Baseline Cost Per Transaction
Valid (avg)	0.000077 ETH
Overcap 5x (avg)	0.000312 ETH
Overcap multiplier	~4.05x cost

The SecuredPaymaster rejects overcap operations in _validatePaymasterUserOp before any on-chain commitment, reducing the cost for these operations to exactly 0 ETH.

5.3 Non-Whitelisted Target Exploitation

In this attack, the adversary sends operations targeting contract addresses not in the paymaster's approved list. Without whitelisting, the paymaster sponsors calls to arbitrary contracts. In the simulation, non-whitelist operations target a signer EOA address, which is a common pattern when an attacker probes the paymaster's scope. Against the BaselinePaymaster, these operations succeed at approximately 0.000075 ETH each. The SecuredPaymaster rejects them at validation with zero cost.

5.4 PostOp Failure

The ERC-4337 spec allows a paymaster's postOp to be called in a separate execution context after the main call. If postOp reverts, EIP-4337 can charge the paymaster the full pre-fund as a penalty. A sophisticated attacker can attempt to engineer conditions that cause postOp to revert, compounding the deposit drain. The BaselinePaymaster's no-op postOp is immune to this specific attack vector by virtue of having no logic, but it provides no protection against the other vectors. The SecuredPaymaster's postOp implementation handles quota reconciliation defensively to avoid revert conditions.

6. Secured Paymaster Design

The SecuredPaymaster implements a defense-in-depth validation pipeline. Each check is independent; a UserOperation must pass all checks to receive sponsorship. The checks are ordered from cheapest to most expensive to minimize gas cost for rejection paths.

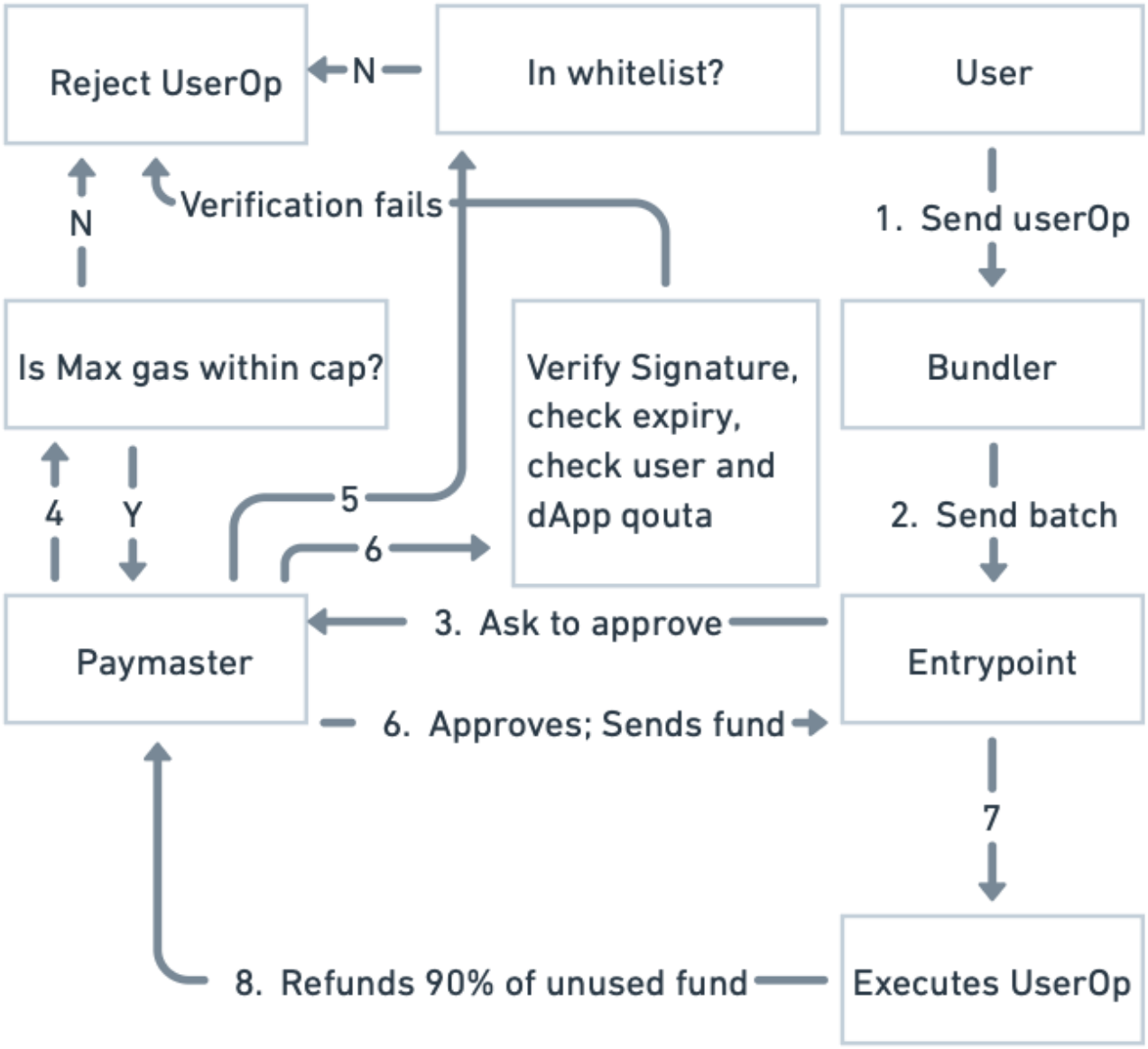

Figure 2. SecuredPaymaster validation pipeline: gas cap → whitelist → signature & quota check → approve / reject.

6.1 Gas Cap Enforcement

The paymaster maintains configurable maximum values for each gas component of the UserOperation: maxVerificationGasLimit, maxCallGasLimit, maxPaymasterVerificationGasLimit, maxPaymasterPostOpGasLimit, and maxPreVerificationGas. During validation, each component is compared against its cap:

```
if (maxVerificationGasLimit != 0) {
    require(
        userOp.unpackVerificationGasLimit() <= maxVerificationGasLimit,
        "SecuredPaymaster: verificationGasLimit exceeds cap"
    );
}
// ... similar checks for callGasLimit, preVerificationGas, etc.
```

A value of zero for any cap disables that particular check, allowing operators to selectively enforce only the gas fields relevant to their deployment. UserOperations exceeding any active cap are rejected with no on-chain cost to the paymaster.

6.2 Contract Whitelisting

The paymaster maintains a mapping(address => bool) allowedTargets storage variable. During validation, the callData of the UserOperation is parsed to extract the target address using the _extractExecuteTarget helper, which validates the 4-byte function selector against the BaselineAccount.execute(address, uint256, bytes) ABI:

```
address target = _extractExecuteTarget(userOp.callData);
require(allowedTargets[target], "SecuredPaymaster: target not allowed");
```

Operations targeting non-whitelisted addresses are rejected at this step. The whitelist is administered by the paymaster owner through the setAllowedTarget(address, bool) function, allowing operators to add and remove approved contracts without redeployment.

6.3 Quotas and Rate Limiting

The paymaster enforces two independent daily spending limits: per-virtual-user and per-dapp (global). User identity is represented as a bytes32 virtualUserId, decoupled from on-chain address identity to support privacy-preserving user management at the off-chain layer.

Daily reset logic uses block.timestamp divided by the seconds-per-day epoch:

```
uint48 currentDay = uint48(block.timestamp / 1 days);
UserDailyQuota storage userQuota = userDailyQuota[virtualUserId];
if (userQuota.dayIndex != currentDay) {
    userQuota.dayIndex = currentDay;
    userQuota.spentWei = 0;
}
require(
    userQuota.spentWei + requiredPreFund <= userDailyLimitWei,
    "SecuredPaymaster: user daily quota exceeded"
);
require(
    dappSpentWei + requiredPreFund <= dappDailyLimitWei,
    "SecuredPaymaster: dapp daily quota exceeded"
);
```

In our experiment, the per-user daily limit was set to 0.01 ETH and the dapp-global limit to 0.1 ETH. User1 was assigned 40 of the 80 valid operations to create concentrated quota pressure and test the reset mechanism.

6.4 Signature Validation and Replay Protection

Every sponsored operation must carry a signed authorization payload embedded in the paymasterAndData field. The payload is encoded as (bytes32 virtualUserId, uint48 validUntil, uint256 sponsorshipNonce, uint256 sponsoredMaxCostWei, uint8 v, bytes32 r, bytes32 s). The sponsorship signature is produced off-chain by a designated signer key and binds the following fields into a digest:

- Paymaster contract address (prevents cross-contract replay)
- Chain ID (prevents cross-chain replay)
- virtualUserId (binds the sponsorship to a specific user identity)
- validUntil (expiry timestamp, set to 3 minutes from issuance in experiments)
- sponsorshipNonce (unique per-authorization nonce)
- sponsoredMaxCostWei (maximum ETH the sponsor commits to covering)

```
bytes32 digest = _sponsorshipDigest(
    virtualUserId, validUntil, sponsorshipNonce, sponsoredMaxCostWei
);
address recoveredSigner = _recoverSigner(digest, v, r, s);
require(recoveredSigner == sponsorSigner,
    "SecuredPaymaster: bad sponsor signature");

require(!usedSponsorshipNonce[sponsorshipNonce],
    "SecuredPaymaster: nonce already used");
```

The usedSponsorshipNonce mapping persists consumed nonces indefinitely, providing unconditional replay protection. Signatures are validated using EIP-191 personal_sign format (\x19Ethereum Signed Message:\n32).

7. System Architecture

7.1 On-Chain Components

The on-chain stack consists of five deployed contracts on the Ethereum Sepolia testnet:

Contract	Address (Sepolia)	Role
EntryPoint	0xc388b22Cc99Db398e6F04a5a39e879dcC6333c5d	ERC-4337 singleton orchestrator
BaselinePaymaster	0xF011e6134eC20eA4Da73e8272C14dfF00Ef61188	Permissive control benchmark
SecuredPaymaster	0x867e56382b806DEB838e4C65a0d64C9597301185	Policy-enforced sponsor

BaselineAccount	0xD4923772777916bf3867BDC6095c13bE64Be8E48	Minimal smart contract wallet
MockTarget	0x97a9b16839d578e33F392654Ba4cF94b944d4E04	Whitelisted target contract

All contracts were compiled with Solidity 0.8.28 with optimization enabled and evmVersion set to 'cancun'. The BaselineAccount uses a deliberately permissive signature validation (always valid) to isolate paymaster-level security behavior from account-level security.

The deployment workflow uses the following Hardhat scripts (see package.json for full set):

```
npm run deploy:entrypoint:sepolia
npm run deploy:baseline:sepolia
npm run deploy:account:sepolia
npm run deploy:secured:sepolia
npm run deploy:mocktarget:sepolia
npm run whitelist:mocktarget:sepolia
```

7.2 Off-Chain Components

The off-chain layer comprises five main components:

- Signing Scripts (TypeScript): runScenarioMatrix_80_10_10.ts and runScenarioMatrix_100.ts generate sponsorship signatures using the deployer's private key, build UserOperation structs, and submit them to the Sepolia bundler via ethers.js. The same randomized operation plan is replayed against both paymasters for controlled comparison.
- Backend Analytics Server (Node.js/Express): backend/server.js parses matrix script terminal output, pulls transaction receipts via ethers.js, extracts UserOperationEvent.actualGasCost from EntryPoint logs, and persists results to CSV and JSON in backend/data/.
- Dashboard Frontend (React + Vite + TypeScript): frontend/src/App.tsx renders live cost, drain, latency, and quota metrics consumed from the backend REST API. Used during experiment runs to monitor scenarios and post-hoc to compare runs.
- Sponsorship Service Scaffold (backend/sponsorship.js): A planned production signing service currently implemented as a placeholder (signature: '0x'). In production, this would be an authenticated API that issues time-limited, nonce-bound signatures per user request.
- Analysis Pipeline (Python): results/paymaster_analysis.py loads userop-results.csv with pandas, computes summary statistics, generates the break-even cost curve, and produces the figures used in this report.

8. Implementation

8.1 Repository Structure

The codebase is organized into five top-level directories to separate concerns between contracts, deployment scripts, off-chain services, analytics, and the experiment dashboard:

```
ERC4337-Paymasters/
├── contracts/                    Solidity sources
│   ├── BaselinePaymaster.sol     Permissive control benchmark
│   ├── SecuredPaymaster.sol      Policy-enforced sponsor
│   ├── BaselineAccount.sol       Minimal IAccount wallet
│   ├── EntryPoint.sol            Re-export from @account-abstraction
│   └── MockTarget.sol            Whitelisted target contract
├── scripts/                      Hardhat + ethers.js scripts
│   ├── deploy*.ts                Per-contract deployment
│   ├── runScenarioMatrix_80_10_10.ts   Mixed-traffic experiment
│   ├── runScenarioMatrix_100.ts        Clean-traffic experiment
│   ├── sendUserOpWith*.ts        Single-op verification scripts
│   └── whitelistSecuredPaymasterMockTarget.ts
├── backend/                      Node.js / Express analytics server
│   ├── server.js                 Parses results, exposes REST API
│   ├── sponsorship.js            Sponsorship signing scaffold
│   └── data/                     CSV + JSON outputs
├── frontend/                     React + Vite dashboard
│   └── src/App.tsx               Live metrics viewer
├── results/                      Analysis artifacts
│   ├── paymaster_analysis.py     Python plotting + stats
│   ├── comparison_summary.csv    Aggregated metrics
│   ├── breakeven_analysis.csv    Cost curves vs. malicious %
│   └── paymaster_analysis.png    Final figure set
├── deployments.json              Deployed Sepolia addresses
├── hardhat.config.ts             Solc + network config
└── Documents/                     Final report + presentation
```

The split between scripts/ and backend/ reflects a deliberate separation: scripts/ holds short-lived experiment runners that talk directly to Sepolia, while backend/ holds the long-running analytics server that reads experiment outputs and serves them to the frontend dashboard. This makes it possible to re-run experiments without touching the dashboard and vice versa.

8.2 Contract Overview

The project contains four primary contracts:

Contract	Purpose
BaselinePaymaster.sol	Permissive paymaster; _validatePaymasterUserOp returns ("", 0) unconditionally.
SecuredPaymaster.sol	Policy-enforced paymaster with gas caps, whitelist, quotas, and signature validation.
BaselineAccount.sol	Minimal IAccount implementation; always-valid signature, EntryPoint-only execution.

MockTarget.sol	Minimal contract deployed as the whitelisted target for valid scenario traffic.

8.3 Key Functions: validatePaymasterUserOp

8.3.1 Baseline Implementation

The baseline implementation trivially satisfies the BasePaymaster interface:

```
function _validatePaymasterUserOp(
    PackedUserOperation calldata,
    bytes32,
    uint256
) internal pure override
    returns (bytes memory context, uint256 validationData)
{
    return ("", 0);
}
```

The return value of (context, validationData) = ("", 0) signals unconditional approval to the EntryPoint with an empty context passed to postOp.

8.3.2 Secured Implementation

The secured implementation performs the full validation pipeline described in Section 6. The complete function signature and quota update logic is as follows:

```
function _validatePaymasterUserOp(
    PackedUserOperation calldata userOp,
    bytes32,
    uint256 requiredPreFund
) internal override returns (bytes memory context, uint256 validationData) {
    // 1. Gas cap checks (Section 6.1)
    // 2. Whitelist check (Section 6.2)
    // 3. Decode paymasterAndData sponsorship payload
    // 4. Expiry + nonce replay check (Section 6.4)
    // 5. Signature verification (Section 6.4)
    // 6. Quota enforcement + update (Section 6.3)
    userQuota.spentWei += requiredPreFund;
    dappSpentWei += requiredPreFund;
    usedSponsorshipNonce[sponsorshipNonce] = true;
    return (abi.encode(virtualUserId, requiredPreFund), 0);
}
```

The context returned encodes (virtualUserId, requiredPreFund) for use by postOp. The validationData of 0 indicates unconditional approval (no time-bound validation through the EntryPoint's native mechanism — expiry is handled explicitly in the logic).

8.4 Key Functions: postOp

The BaselinePaymaster's postOp is a no-op. The SecuredPaymaster's postOp performs quota reconciliation:

```
function _postOp(
    PostOpMode mode,
    bytes calldata context,
    uint256 actualGasCost,
    uint256
) internal override {
    (bytes32 virtualUserId, uint256 reservedCostWei) =
        abi.decode(context, (bytes32, uint256));
    if (actualGasCost < reservedCostWei) {
        uint256 refund = reservedCostWei - actualGasCost;
        userDailyQuota[virtualUserId].spentWei -= refund;
        dappSpentWei -= refund;
    }
}
```

If the actual gas cost is lower than the pre-fund reservation, the difference is refunded back to both the user's and dapp's daily quota counters. This ensures that quota consumption accurately reflects actual spending rather than worst-case pre-fund estimates.

8.5 Implementation Challenges and Resolutions

Several non-trivial issues surfaced during implementation. Each is documented below with its resolution to make the engineering effort reproducible.

Challenge	Resolution
paymasterAndData layout	The packed UserOperation format places the paymaster address in the first 20 bytes of paymasterAndData, followed by paymaster verification and post-op gas limits (32 bytes), with sponsorship payload starting at byte offset 52. Initial implementations decoded from offset 20, producing malformed signature recovery. Resolution: standardized on userOp.paymasterAndData[52:] for ABI-decoding the sponsorship payload.
EIP-191 digest construction	Off-chain signing initially used the raw keccak256 hash, but on-chain ECDSA recovery expects the EIP-191 prefix. Resolution: implemented _toEthSignedMessageHash inside _sponsorshipDigest and matched it in the TypeScript signer with ethers.signMessage.
Gas estimation under Sepolia volatility	Sepolia's gas price oscillates more than mainnet, occasionally producing requiredPreFund values that exceeded the per-user daily quota for legitimate users. Resolution: added a 20% gas-price safety margin in the matrix scripts and tuned the per-user limit to 0.01 ETH to comfortably absorb fluctuations.
Bundler simulation rejections	Early scenario runs hit AA33 reverted: SecuredPaymaster errors during bundler

	simulation, which silently dropped operations from the mempool. Resolution: added explicit revert reason parsing in the matrix scripts to log rejected operations into the CSV with result=reverted, allowing them to be counted against attack-blocking statistics.
Quota refund accuracy	The first version of postOp did not reconcile actual cost vs. pre-fund, causing the user quota to deplete based on worst-case estimates. Resolution: rewrote _postOp to refund the difference (reservedCostWei - actualGasCost) back to both userQuota.spentWei and dappSpentWei.
Whitelist target extraction	callData decoding initially accepted any 4-byte selector, allowing operations whose selector did not match BaselineAccount.execute to bypass whitelisting silently. Resolution: _extractExecuteTarget now reverts on selector mismatch, forcing all sponsored ops through the validated execute(address,uint256,bytes) path.

9. Methodology

9.1 Test Setup

All experiments were conducted on the Ethereum Sepolia testnet using a dedicated deployer account. Both paymasters were funded with 0.05 ETH at the EntryPoint deposit prior to each scenario run. The Hardhat development environment was configured with the Alchemy Sepolia RPC endpoint and a dedicated deployer private key.

The technology stack is summarized as follows:

Component	Technology
Smart Contract Language	Solidity 0.8.28
Development Framework	Hardhat + TypeScript
AA Library	@account-abstraction/contracts v0.8
EVM Library	ethers.js v6
Network	Ethereum Sepolia Testnet
RPC Provider	Alchemy
EVM Version	Cancun
Dashboard Backend	Node.js / Express

Dashboard Frontend	React + Vite + TypeScript
Analytics	Python (pandas, matplotlib)
Data Storage	CSV + JSON (local filesystem)

Hardhat is configured for Solidity 0.8.28 with the Cancun EVM and the Sepolia network:

```
// hardhat.config.ts
solidity: {
  version: "0.8.28",
  settings: {
    optimizer: { enabled: true, runs: 200 },
    evmVersion: "cancun"
  }
},
networks: {
  sepolia: {
    url: process.env.SEPOLIA_RPC_URL,
    accounts: [process.env.DEPLOYER_PRIVATE_KEY]
  }
}
```

9.1.1 Tool and Technology Justification

Tool choices and their main rationale:

Choice	Rationale
Hardhat (vs. Foundry)	TS-native scripting and ethers.js integration; needed for orchestrating off-chain signing + on-chain calls.
Solidity 0.8.28 / Cancun EVM	Required by @account-abstraction/contracts v0.8 (transient storage).
@account-abstraction/contracts	Canonical, audited EntryPoint matching production deployments.
ethers.js v6	Native PackedUserOperation support and mature AA tooling ecosystem.
Sepolia testnet	Realistic bundler/mempool conditions at zero cost; mainnet rejected on cost grounds.
Alchemy RPC	Reliable rate limits for high-volume scenario runs.
Node.js + Express backend	Reuses ethers.js types from scripts; Python used only for post-hoc analytics.
React + Vite frontend	Minimal config for a local single-page metric dashboard.
CSV + JSON storage	Append-only experiment data, human-inspectable, version-controllable.

9.2 Traffic Scenarios

Two distinct traffic mixes were tested, each replayed against both the Baseline and Secured paymasters with identical operation sequences:

Label	Mix	Total Ops	Description
Scenario A	80-10-10	200 total ops	80% valid, 10% overcap (5x gas), 10% non-whitelist. User1 assigned 40 valid ops for quota pressure.
Scenario B	100% Valid	200 total ops	100% valid operations with randomized user assignment. No attack traffic.

For the overcap simulation, normal gas profile values were multiplied by 5x. Non-whitelist operations were directed at the deployer EOA address (not in the whitelist). The same randomized assignment was used for both paymasters in Scenario A to ensure a controlled comparison. Scenario B provides a clean-traffic baseline to measure the overhead cost of the secured validation pipeline with no benefit.

9.3 Metrics Collection

The following metrics were collected for each UserOperation:

- sponsor_drain_eth: ETH deducted from paymaster deposit, sourced from UserOperationEvent.actualGasCost emitted by the EntryPoint.
- txn_cost_eth: Total on-chain transaction cost, computed as gasUsed * effectiveGasPrice from the transaction receipt.
- result: Success or failure of the UserOperation execution.
- user_quota / dapp_quota: Remaining daily quota after each operation (SecuredPaymaster only).
- Execution time: Wall-clock time per operation, measured in the TypeScript scripts.

Data was stored in both JSON (userop-history.json) and CSV (userop-results.csv) formats for analysis. The CSV schema is: type, case, txn_hash, sponsor_drain_eth, txn_cost_eth, result, user_quota, dapp_quota.

The CSV (userop-results.csv) follows this schema:

Field	Type	Example	Description
type	string	baseline / secured	Paymaster variant used
case	string	valid / overcap / non-whitelist	Operation scenario type
txn_hash	string (0x...)	Sepolia txn hash	On-chain transaction hash
sponsor_drain_eth	float	e.g. 0.000095	ETH deducted from paymaster deposit

txn_cost_eth	float	e.g. 0.000110	Total txn cost (gasUsed * effectiveGasPrice)
result	string	success / reverted	UserOperation execution result
user_quota	float	Remaining ETH in quota	User daily quota remaining (Secured only)
dapp_quota	float	Remaining ETH in quota	Dapp daily quota remaining (Secured only)

10. Results

10.1 Cost Analysis: Total Sponsor Drain

Table 3 and Figure 4 summarize the total sponsor drain (ETH deducted from paymaster deposit) for each scenario and paymaster variant.

Metric	Scenario A (80-10-10)	Scenario B (100% Valid)
Baseline — Total	0.010056 ETH	0.006905 ETH
Baseline — Per-Txn Avg	0.000101 ETH	0.000069 ETH
Secured — Total	0.009541 ETH	0.011144 ETH
Secured — Per-Txn Avg	0.000095 ETH	0.000111 ETH
Delta (Secured saves)	+0.000515 ETH (+5.1%)	-0.004239 ETH (-61.4%)
Winner	Secured ✓	Baseline ✓

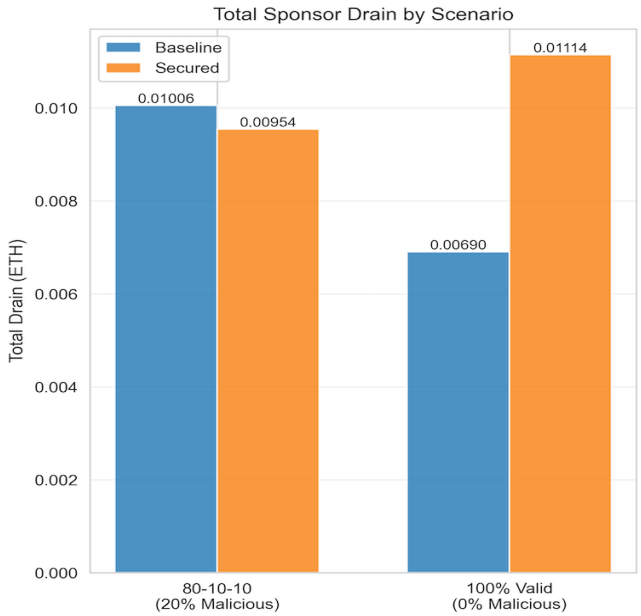

Figure 4. Total sponsor drain by scenario — Baseline vs. Secured Paymaster, 80-10-10 mix and 100% valid mix (200 ops each).

In Scenario A (20% malicious traffic), the Secured Paymaster achieves a 5.1% reduction in total drain by completely eliminating costs from overcap and non-whitelist operations. In Scenario B (0% malicious traffic), the Secured Paymaster incurs 61.4% higher drain per-transaction due to the validation overhead cost with no offsetting attack prevention benefit.

10.2 Cost Analysis: Breakdown by Scenario Type

Figure 5 and Table 4 decompose the per-transaction cost for the 80-10-10 scenario by operation type.

Operation Type	Baseline Avg Cost	Secured Avg Cost	Delta
VALID (80 ops)	0.000077 ETH	0.000119 ETH	+54.5% overhead
OVERCAP (10 ops)	0.000312 ETH	0.000000 ETH	100% blocked
NON-WHITELIST (10 ops)	0.000075 ETH	0.000000 ETH	100% blocked

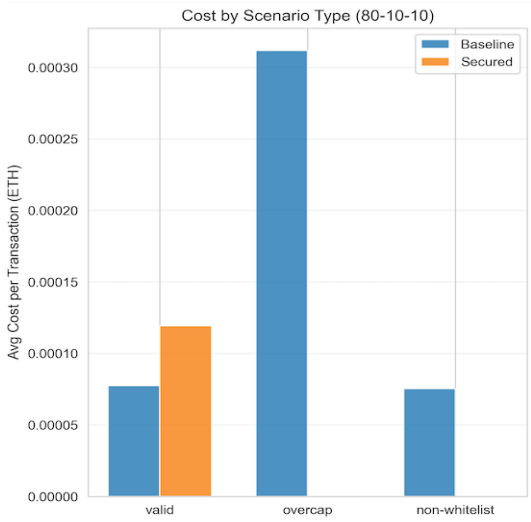

Figure 5. Per-transaction cost by operation type (80-10-10): Secured drops overcap and non-whitelist costs to zero while adding overhead on valid ops.

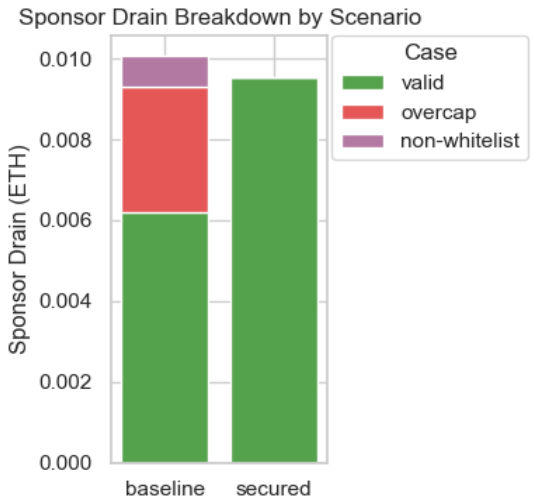

Figure 5b. Sponsor drain breakdown — Baseline drain stacks valid + overcap + non-whitelist contributions; Secured drain is composed only of valid traffic.

The Secured Paymaster achieves zero cost on both attack categories (overcap and non-whitelist), confirming that the validation pipeline completely blocks the simulated exploits at the pre-execution stage. Valid operations carry a per-transaction overhead of approximately 0.000042 ETH due to the additional validation computation.

10.3 Performance Analysis: Execution Time

Paymaster	Total Time	Per-Txn Time	Comparison
Baseline	1572.8 s (total)	15.73 ms/txn	—
Secured	1353.1 s (total)	13.53 ms/txn	14.0% faster
Baseline	1320.2 s (total)	13.20 ms/txn	—
Secured	1465.9 s (total)	14.64 ms/txn	10.9% slower

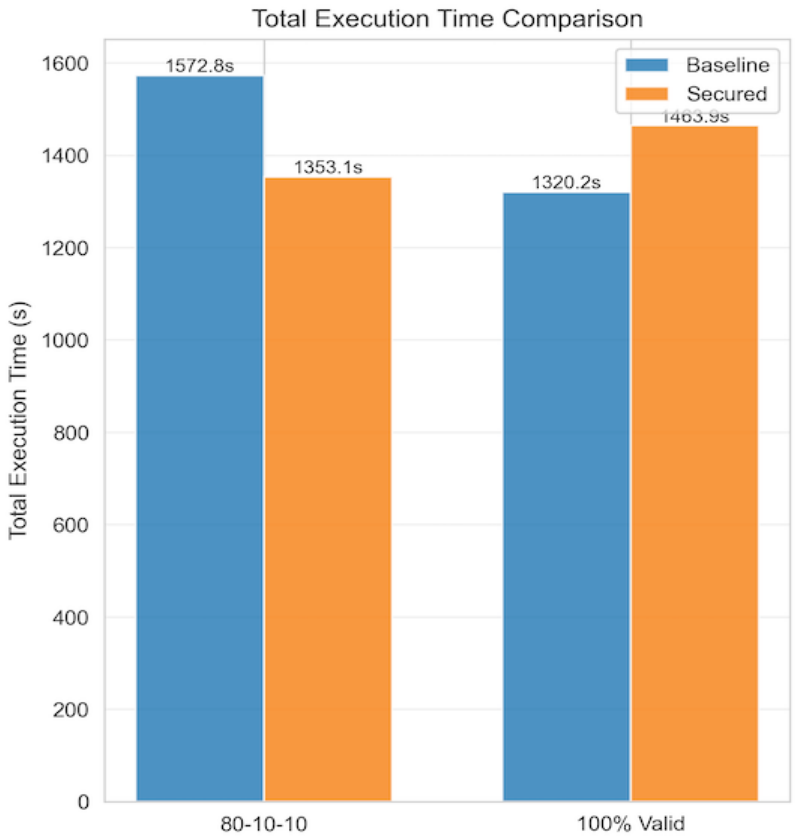

Figure 6. Total execution time comparison — Secured is 14% faster on 80-10-10 (fail-fast on attacks) and 10.9% slower on 100% valid (validation overhead with no rejections).

In Scenario A, the Secured Paymaster is 14% faster overall despite adding validation overhead on valid operations. This is because early rejection of 20 malicious operations (each of which would have proceeded to on-chain execution under the baseline) saves more time than validation adds to the 80 valid operations. The fail-fast property of early rejection is a key performance advantage in attack conditions.

In Scenario B (no attack traffic), the Secured Paymaster is 10.9% slower, as all 100 operations incur full validation overhead with no compensating rejections.

10.4 Tradeoffs Summary

The empirical results confirm the fundamental tradeoff: the Secured Paymaster imposes a fixed overhead on every valid transaction in exchange for complete elimination of attack costs. This overhead is justified only when attack traffic exceeds the break-even threshold derived in Section 11.

Important caveat — these results depend on the assumed traffic mix and volume. The 5.1% savings, the per-transaction overhead, and the 17.87% break-even point are all derived from a 200-operation experiment with overcap multipliers of 5x and a specific gas profile. In production, the cost picture varies sharply with both the intensity of the attack (gas-limit multiplier, how aggressively quotas are abused) and the absolute number of malicious operations submitted.

A single high-intensity attack — even at low total volume — can drain a large share of a paymaster's deposit before any rate-limit signal is observed; conversely, a sustained low-intensity attack can drain it slowly without ever crossing a per-operation alarm threshold. Operators should re-derive these thresholds against their own observed traffic distribution rather than treating the 17.87% figure as universal.

11. Break-Even Analysis

11.1 Derivation

The break-even point is the percentage of malicious traffic at which the Secured Paymaster's savings from attack prevention equal its additional cost from validation overhead. Define:

- validation_overhead = 0.000042 ETH (per valid transaction overhead)
- avg_malicious_baseline_cost = 0.000193 ETH (weighted avg cost of overcap + non-whitelist ops under Baseline)
- secured_malicious_cost = 0.000000 ETH (Secured blocks all malicious ops)

The break-even formula is:

```
breakeven_pct = (validation_overhead /
                (avg_malicious_baseline_cost + validation_overhead)) * 100

breakeven_pct = (0.000042 / (0.000193 + 0.000042)) * 100
              = 17.87%
```

At 17.87% malicious traffic, the cost of Secured validation overhead on valid operations exactly equals the cost savings from blocking malicious ones. Above this threshold, the Secured Paymaster is economically advantageous.

11.2 Decision Matrix

Malicious Traffic %	Cost Winner	Speed Winner	Recommendation
0–10%	Baseline	Baseline	Overhead not justified; clean traffic best served by low-latency baseline.
11–17%	Monitor	Mixed	Approaching break-even; begin monitoring attack rate trends.
18–30%	Secured	Secured	Both cost and speed favor Secured at this attack rate.
>30%	Secured (strongly)	Secured (strongly)	Secured provides strong ROI; hybrid routing may still help for verified users.

11.3 Economic Projection

Assuming 100,000 transactions per month with 20% malicious traffic and the empirically observed savings rate of 0.0008 ETH per 100 transactions:

```
Monthly savings = (100,000 / 100) * 0.0008 ETH
                = 0.8 ETH/month
                ≈ $1,600/month (at $2,000/ETH)
```

This projection assumes ETH price of approximately $2,000 USD. The actual savings scale linearly with transaction volume and ETH price.

12. Deployment Strategy

12.1 When to Use Secured vs Baseline

Based on the break-even analysis, the Secured Paymaster is recommended when:

- Network malicious traffic is at or above the 18% threshold (using the experimental dataset as reference).
- Gas-cap, whitelist, and quota enforcement are contractual or compliance requirements for the sponsoring protocol.
- The paymaster deposit is at risk from known adversarial actors who have demonstrated exploit capability.
- The operator cannot monitor and manually revoke individual addresses quickly enough to prevent drain.

The Baseline Paymaster may be appropriate when:

- The operator is in a fully closed environment where all senders are verified (e.g., enterprise internal transactions).

- Traffic is entirely clean and the operator has strong off-chain verification before UserOps reach the paymaster.
- The per-transaction overhead of ~0.000042 ETH is unacceptable for the use case (e.g., very high-frequency micro-transactions).

12.2 Hybrid Deployment

A recommended production architecture routes traffic dynamically based on risk signals:

```
Incoming UserOperation
    |
    |--> [Off-chain Risk Scoring]
           |
           |--> Score < threshold --> BaselinePaymaster (known-good user)
           |
           |--> Score >= threshold --> SecuredPaymaster (unknown/risky user)
```

Risk scoring inputs can include address age, historical transaction success rate, virtual user ID reputation score, and membership in a verified user registry. This hybrid approach captures the low overhead of the Baseline for trusted users while protecting against unknown or adversarial actors with the Secured path. The key implementation requirement is an off-chain router that can make the paymaster selection decision before UserOp submission.

12.3 Operational Considerations

- Attack rate monitoring: Deploy dashboards that track the ratio of rejected to accepted operations over time. Alert when the rejection rate approaches the break-even threshold from either direction.
- Quota calibration: Set daily per-user limits based on the 95th percentile (or higher) of legitimate user gas consumption, not on attack scenarios. Over-conservative limits will create false positives for power users.
- Signer key management: The sponsorship signing key must be protected with HSM or MPC-based custody. A compromised signing key invalidates all signature-based security guarantees.
- Whitelist governance: Maintain an on-chain governance process (multi-sig or timelock) for whitelist modifications to prevent admin key compromise from instantly opening the paymaster to arbitrary targets.

13. Limitations

The Secured Paymaster's design comes with explicit tradeoffs. Validation adds approximately 10.9% latency overhead on clean traffic and increases per-op verification gas, which feeds into requiredPreFund. Signature validation creates a hard dependency on the off-chain signing service, which in this project is a placeholder; production deployment requires HA infrastructure with HSM-backed key management. Fixed daily quotas can produce false positives for high-volume legitimate users and need calibration against observed traffic. The signature scheme uses EIP-191 personal_sign rather than typed EIP-712 data, which is weaker for cross-application replay protection. Whitelist target extraction assumes the BaselineAccount.execute(address,uint256,bytes) ABI, creating a hidden coupling between paymaster and account implementations. Finally, all measurements were taken on Sepolia; absolute ETH values do not reflect mainnet EIP-1559

dynamics, but the percentage-based results (e.g., 17.87% break-even, 5.1% savings at 80-10-10) are expected to generalize.

14. Future Work

Several extensions would materially strengthen the framework. Adaptive controls — dynamic gas caps tracked from rolling means, automated quota scaling tied to user reputation, and ML-based anomaly detection at the bundler layer — would let static thresholds evolve with attack patterns. Decentralized authorization (k-of-n multi-party signing, ZK-proof-based eligibility, or on-chain reputation oracles such as EAS / Gitcoin Passport) would remove the single-key trust assumption. Migrating from EIP-191 to EIP-712 typed structured data would bind signatures to a specific paymaster's domain and prevent cross-contract replay even under shared keys. The placeholder sponsorship signing service should be replaced with an authenticated, rate-limited, HSM-backed production service with audit logging. Finally, a comparative study against production paymasters (Biconomy, Alchemy, ZeroDev) and post-mortems of real-world incidents would validate whether the 17.87% break-even threshold holds at production scale.

15. Conclusion

Paymasters are among the highest-value targets in the ERC-4337 stack — they hold ETH, they're publicly addressable, and a naive implementation will sponsor anything sent to them. This project built the case, in measurable terms, for why that matters and what it costs to fix.

Running identical attack traffic against both a permissive and a hardened paymaster on Sepolia, the SecuredPaymaster eliminated all simulated exploit costs while adding roughly 0.000042 ETH of overhead per valid transaction. That overhead pays for itself when malicious traffic crosses ~18% of total volume — a threshold that's not hard to reach in a live, open deployment.

Below that threshold, the overhead isn't worth it. That's not a flaw in the design; it's the honest tradeoff. A hybrid routing strategy — sending known-good users through the baseline and unknown or high-risk senders through the secured path — is likely the right answer for most production operators.

The numbers here (5.1% savings at 20% attack traffic, 0.8 ETH/month projected at 100k ops) are specific to the experimental setup and shouldn't be treated as universal. What does generalize is the method: measure your own traffic mix, derive your own break-even, and make the deployment decision from data rather than assumption. The contracts, scripts, and analysis pipeline in this repo are built to make that straightforward.